

Design: NETransparentProxyProvider for durable, launch-method-agnostic Copilot desktop interception

2026-10-02. This closes the one gap left after tonight's live validation of the model-catalog-faking approach (see `copilot-desktop-routing-approaches-comparison.md` and the mitm capture work in this session): every interception technique proven tonight (`HTTPS_PROXY` tunneling, the real `/models` capture, the live `{"data": []}` fake that durably kept the real 28 models out of `app_state`) only takes effect because we controlled how the process was launched. A user double-clicking the real Dock icon bypasses all of it. This document is the design for the one mechanism that doesn't have that gap.

What was actually proven tonight, and what's still missing

Confirmed live, this session: - The Copilot desktop app's real model catalog comes from `GET https://api.business.githubcopilot.com/models` (plain GitHub OAuth bearer, `copilot-harness-id: copilot-sdk`, `copilot-integration-id: copilot-developer-app` — a different identity than `listener::kinds:: copilot_desktop` 's current injected headers). - The app's native Rust backend reads `HTTPS_PROXY / HTTP_PROXY` **environment variables only** — confirmed by direct contrast: a system -wide GUI proxy change captured the Electron/web-layer calls (`copilot_internal/user` , telemetry) but never the backend's `/models` call; a **process-scoped env var** did. - Faking that one endpoint's response to `{"data": []}` — via a `mitmproxy` addon, reached by launching the real binary directly with `HTTPS_PROXY` set for that one process — durably kept `app_state.copilot-cloud- available-models-v2` at one entry (our own custom provider) through a real app launch. No DB-level fight with the account's live resync at all; the resync just has nothing real to restore from, because the source response itself was faked.

What's missing: all of this only happens when we set the env var at launch time, by hand, via a direct binary invocation. There is no durable way to inject that env var into the app's launch on macOS short of a wrapper script the user must choose to run instead of the real icon (`Info.plist / LSEnvironment` edits and binary-swap tricks both break code signing and get wiped by the app's own auto-update — already ruled out earlier this session). `NETransparentProxyProvider` is the one real OS primitive that removes that dependency entirely.

Why this provider type specifically, not the other two

Three Network Extension provider types exist on macOS; only one fits.

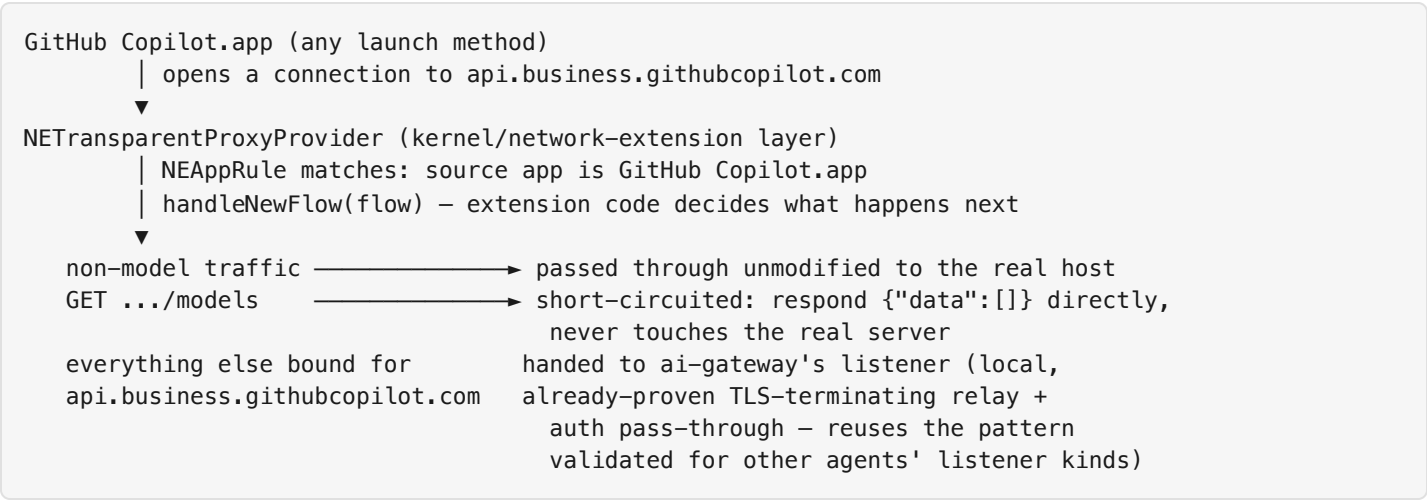
Provider	Can redirect?	Scoped to one app?
<code>NEFilterDataProvider</code> (what the existing, unbuilt <code>network_egress</code> macOS design targets — <code>src/services/net_egress/macos.rs</code>)	No — allow/drop only	N/A
<code>NEDNSProxyProvider</code>	Can fake a DNS answer dynamically	No — answers DNS for every process querying that hostname, same collateral-scope problem as a static <code>/etc/hosts</code> entry

Provider	Can redirect?	Scoped to one app?
NETransparentProxyProvider	Yes — hands a matched flow to the extension as a proxyable object	Yes — NEAppRule matches by the <i>source app</i> 's bundle identifier

NEAppRule's app-scoping is the whole reason this is the right primitive: a rule of the shape "any flow whose source app is the GitHub Copilot desktop app → hand to our extension" is enforced **below the process-launch layer entirely**. It does not care whether the app was started from the Dock, Spotlight, Finder, or a shell — there is no launch-time hook to route around, because the interception point is the kernel's network stack, not the process's own environment.

This is the same provider type `option-4-process-aware-redirect-design.md` already identified for the general "redirect a brokered agent's traffic" goal (that doc's target was the Microsoft 365 Copilot client, a different agent). This document is the Copilot-desktop-specific instance of that same primitive, now motivated by a concretely-validated need rather than a general capability gap, and scoped to the one thing proven necessary tonight: durably replacing a one-shot `HTTPS_PROXY` env var with something that holds on every real-world launch.

Architecture



Three behaviors, not one blanket redirect: 1. **The model-catalog endpoint is faked**, exactly as validated tonight — `GET /models` never reaches the real backend at all. 2. **Everything else bound for the real inference/account hosts is handed to the existing (or a new) listener kind**, which does what `listener::kinds::copilot_desktop` already does for the BYOK path: pass the client's own real `Authorization` through unchanged (the app's first-party traffic already carries a real bearer, confirmed this session — unlike BYOK, nothing needs injecting), optionally broker through Optimus, and relay the real response back. 3. **Everything unrelated to Copilot** (`api.github.com` account/auth calls, telemetry, the general `alive.github.com` notification socket) is left alone — the whole point of app-scoped matching is that it only ever sees this one app's flows in the first place, so there's no need for a second filter to carve those out.

What shipping this actually requires

- **A new, separate signed system-extension bundle.** Not an extension of the existing Endpoint Security extension (`endpoint-ext/` , ES-based, no NE code today) and not the same target as the still-unbuilt `NEFilterDataProvider` design `net_egress/macos.rs` already has daemon -side plumbing for — a third extension target in the product.
- **Its own entitlement.** The real, Apple-issued provisioning profile already found this session (`packaging/macos/profiles/ network.provisionprofile` , dated 2026-07-01) carries `com.apple.developer.networking.networkextension` with several capabilities already — confirm whether the specific app-proxy-provider capability is among them, or whether this needs its own entitlement request. Do not assume the existing grant covers it; it was requested for the content-filter use case.
- **User or MDM approval on first activation.** There's a real, concrete precedent for *silent* MDM approval on this exact class of device: the Zscaler Tunnel extension (`com.zscaler.zscaler.TRPTunnel` , discovered this session via `profiles / system_profiler`) is delivered exactly this way — `com.apple.vpn.managed` , no user click. Whether this product's own extension can piggyback on the same silent-approval path is a question for whoever owns this org's MDM policy, not something to assume.
- **The extension code itself:** a `NETransparentProxyProvider` subclass implementing `handleNewFlow / handleNewUDPFlow` , checking the flow's source app identity, and either answering directly (the `/models fake`) or proxying the flow to the local listener via `NWConnection` .
- **The net-agent helper's command surface needs to grow.** The existing Swift helper (`packaging/macos/agent-app/main.swift`) only implements `activate / deactivate` for the ES extension today — it has no `enable/disable/configure` verbs for any NE provider yet, per the earlier survey of this codebase's macOS groundwork.

Open questions / risks

- **The documented `NEFilterDataProvider / NETransparentProxyProvider` `WebSocket-upgrade` compatibility bug** (`option-4-process-aware-redirect -design.md` 's own finding: both provider types matching the same connection broke `wss://` specifically on macOS 11.1, fixed in 11.2). Not expected to reproduce on any realistically-targeted fleet, but this app has at least one real WebSocket connection (`alive.github.com` 's notification socket) that would be in scope if its filtering rules ever overlapped with this extension's — keep them disjoint by design, per that doc's own defense-in-depth recommendation.
- **Whether the NE entitlement grant already covers app-proxy-provider, or only content-filter** — unverified, stated as a risk rather than assumed either way.
- **MDM silent-approval is observed for a *different* vendor's extension on this device, not proven available to this product's own bundle id.** Don't treat the Zscaler precedent as a guarantee.
- **The `/models` -faking logic itself needs to move out of the `mitmproxy` proof-of-concept and into real, maintained code** — today it's a throwaway Python addon (`ai-gateway/scratchpad/copilot-mitm-body-logger.py`), useful for tonight's validation, not something to ship.

Recommended build order

1. Confirm the exact entitlement capability needed vs. what the existing provisioning profile already grants — before writing any extension code, since this gates whether a new Apple request is even needed.
2. Build the extension with `handleNewFlow` doing only the `/models fake` and pass-through for everything else (no Optimus brokering yet) — the smallest slice that reproduces tonight's validated behavior but durably,

regardless of launch method.

3. Extend `net-agent` with real enable/disable verbs for this provider.
4. Only then layer in brokering the non- `/models` traffic through the existing listener/Optimus path — that's additive once the extension itself is proven to intercept reliably.